PART I - FOUNDATION PLC PROJECTS

Project 8 - Traffic Light Controller

Version 1 - Foundation PLC Project

PLC Platform: Allen-Bradley MicroLogix 1100

Programming Software: RSLogix Micro Starter Lite 8.30

Prepared By: Sharmin Rahman

1. Project Overview

The Traffic Light Controller was developed to introduce cyclic step-based sequencing using internal PLC state bits, timers, and discrete outputs.

The current Version 1 implementation uses B3:0/0 Cycle_Active as the overall cycle enable and three internal step bits: B3:0/1 Green_Step, B3:0/2 Yellow_Step, and B3:0/3 Red_Step. The sequence begins in Red, transitions to Green, then Yellow, and returns to Red continuously while Cycle_Active remains true.

Red and Green each operate for 10 seconds, while Yellow operates for 5 seconds. A Stop command clears Cycle_Active and all step bits. The Red Light is also energized whenever Cycle_Active is false, providing a defined stopped/default indication.

This project expanded my PLC foundation by combining operator start/stop control, step initialization, timer-driven transitions, repeated cyclic sequencing, and a default-state output.

2. Engineering Objective

The objective of this project was to develop and verify a PLC-based traffic light sequence that automatically cycled through Red, Green, and Yellow states with defined timing while providing a clear stopped/default Red indication.

- Latch the traffic-light cycle from a Start command.
- Clear the cycle and all sequence steps from a Stop command.
- Initialize the Red step when the cycle starts and no other step is active.
- Keep the Red Light ON for 10 seconds.
- Transition automatically from Red to Green.
- Keep the Green Light ON for 10 seconds.
- Transition automatically from Green to Yellow.
- Keep the Yellow Light ON for 5 seconds.
- Transition automatically from Yellow back to Red and repeat.
- Keep the Red Light ON whenever the cycle is stopped.

3. Functional Requirements

- Start input I:1/0, with Stop input I:1/1 not active, shall latch Cycle_Active B3:0/0.
- Stop input I:1/1 shall unlatch Cycle_Active and all three step bits.
- When Cycle_Active is true and no step bit is active, Red_Step B3:0/3 shall latch.
- Red Light O:2/2 shall energize when Red_Step is active or when Cycle_Active is false.
- Red Timer T4:0 shall time the Red step for 10 seconds.
- When T4:0/DN becomes true, Red_Step shall unlatch and Green_Step shall latch.
- Green Light O:2/0 shall energize while Green_Step is active.
- Green Timer T4:1 shall time the Green step for 10 seconds.
- When T4:1/DN becomes true, Green_Step shall unlatch and Yellow_Step shall latch.
- Yellow Light O:2/1 shall energize while Yellow_Step is active.
- Yellow Timer T4:2 shall time the Yellow step for 5 seconds.
- When T4:2/DN becomes true, Yellow_Step shall unlatch and Red_Step shall latch.
- The Red -> Green -> Yellow -> Red sequence shall repeat while Cycle_Active remains true.
- Activating Stop at any point shall clear the cycle/steps and leave the Red Light energized through the stopped/default branch.

4. System Components

Component	Identification	Function
PLC Platform	Allen-Bradley MicroLogix 1100	Executes the traffic-light sequence
Start Input	I:1/0	Starts/latches the cycle
Stop Input	I:1/1	Stops the cycle and clears steps
Cycle Active	B3:0/0	Overall sequence enable
Green Step	B3:0/1	Represents Green state
Yellow Step	B3:0/2	Represents Yellow state
Red Step	B3:0/3	Represents Red state
Green Light	O:2/0	Green indication
Yellow Light	O:2/1	Yellow indication
Red Light	O:2/2	Red indication and stopped/default output
Red Timer	T4:0	Times Red state
Green Timer	T4:1	Times Green state
Yellow Timer	T4:2	Times Yellow state
Programming Software	RSLogix Micro Starter Lite 8.30	Ladder logic development and monitoring

5. I/O and Data Assignment

Physical I/O

Device	PLC Address	Type	Description
Start	I:1/0	Digital Input	Starts traffic-light cycle
Stop	I:1/1	Digital Input	Stops cycle and clears steps
Green Light	O:2/0	Digital Output	Green indication
Yellow Light	O:2/1	Digital Output	Yellow indication
Red Light	O:2/2	Digital Output	Red indication

Internal Sequence Bits

Function	Address	Purpose
Cycle Active	B3:0/0	Enables the sequence
Green Step	B3:0/1	Active Green state
Yellow Step	B3:0/2	Active Yellow state
Red Step	B3:0/3	Active Red state

Timer Assignment

Timer	Time Base	Preset	Duration	Function
T4:0	0.01 s	1000	10.00 s	Red
T4:1	0.01 s	1000	10.00 s	Green
T4:2	0.01 s	500	5.00 s	Yellow

6. Control Strategy

I used a cyclic state-based control strategy in which one internal step bit represents the active traffic-light state.

Start latches Cycle_Active, while Stop unlatches Cycle_Active and all three step bits. When Cycle_Active is true and no step is selected, the initializer latches Red_Step.

Each active state directly controls its light output and enables its dedicated timer. When a timer Done bit becomes true, the current step is unlatched and the next step is latched.

The Red Light rung also contains a branch that energizes Red whenever Cycle_Active is false. This gives the stopped system a defined Red indication rather than leaving all lights OFF.

7. Engineering Design Decisions

Design Decision	Reason
Used Cycle_Active internal bit	Separated operator run state from individual traffic-light outputs
Used dedicated B3 step bits	Made each traffic-light state explicit and easy to monitor
Initialized Red when no step was active	Established a known first state after starting
Used separate timer for each state	Made timing values independent and easy to verify
Used timer Done bits for transitions	Provided automatic sequence progression
Cleared all step bits on Stop	Returned the sequence to a known condition
Energized Red when Cycle_Active is false	Provided a clear stopped/default traffic indication
Used 10 s / 10 s / 5 s timing exactly as shown	Preserved the current final ladder implementation

The current implementation is intentionally simple. It does not include opposing-road signals, pedestrian logic, flashing modes, fault monitoring, or safety-rated conflict detection.

8. Ladder Logic Description

The current ladder contains twelve functional rungs, Rung 0000 through Rung 0011, plus the END rung. The representations below follow the approved single-line portfolio format.

8.1 Rung 0000 - Start / Latch Cycle Active

```
|----[ XIC I:1/0 ]----[ XIO I:1/1 ]-----------------------( OTL B3:0/0 )----|
       Start              Stop                              Cycle_Active
```

When Start is true and Stop is not active, Cycle_Active B3:0/0 is latched.

8.2 Rung 0001 - Stop / Clear Cycle and Steps

```
|----[ XIC I:1/1 ]------------------------+----( OTU B3:0/0 )----|
       Stop                              |      Cycle_Active
                                         +----( OTU B3:0/1 )----|
                                         |      Green_Step
                                         +----( OTU B3:0/2 )----|
                                         |      Yellow_Step
                                         +----( OTU B3:0/3 )----|
                                                Red_Step
```

Stop clears Cycle_Active and all three step bits.

8.3 Rung 0002 - Initialize Red Step

```
|----[ XIC B3:0/0 ]----[ XIO B3:0/1 ]----[ XIO B3:0/2 ]----[ XIO B3:0/3 ]----( OTL B3:0/3 )----|
      Cycle_Active       Green_Step         Yellow_Step         Red_Step           Red_Step
```

When the cycle is active and no step is selected, Red_Step is latched.

8.4 Rung 0003 - Red Light Control

```
|----+----[ XIC B3:0/3 ]----+-------------------------------( OTE O:2/2 )----|
     |     Red_Step          |                                  Red_Light                    |
     +----[ XIO B3:0/0 ]-----+
          Cycle_Active
```

Red Light O:2/2 is energized when Red_Step is active or when Cycle_Active is false.

8.5 Rung 0004 - Red Timer

```
|----[ XIC B3:0/3 ]-----------------------[ TON T4:0 ]----|
       Red_Step                          Red_Timer   PRE: 1000   Time Base: 0.01
```

T4:0 times the Red state for 10 seconds.

8.6 Rung 0005 - Transition Red to Green

```
|----[ XIC T4:0/DN ]---------------------+----( OTU B3:0/3 )----|
       Red_Timer_Done                    |      Red_Step
                                         +----( OTL B3:0/1 )----|
                                                Green_Step
```

When Red Timer completes, Red_Step is cleared and Green_Step is latched.

8.7 Rung 0006 - Green Light Control

```
|----[ XIC B3:0/1 ]-----------------------( OTE O:2/0 )----|
      Green_Step                           Green_Light
```

Green Light O:2/0 is energized while Green_Step is active.

8.8 Rung 0007 - Green Timer

```
|----[ XIC B3:0/1 ]-----------------------[ TON T4:1 ]----|
       Green_Step                         Green_Timer   PRE: 1000   Time Base: 0.01
```

T4:1 times the Green state for 10 seconds.

8.9 Rung 0008 - Transition Green to Yellow

```
|----[ XIC T4:1/DN ]---------------------+----( OTU B3:0/1 )----|
       Green_Timer_Done                  |      Green_Step
                                         +----( OTL B3:0/2 )----|
                                                Yellow_Step
```

When Green Timer completes, Green_Step is cleared and Yellow_Step is latched.

8.10 Rung 0009 - Yellow Light Control

```
|----[ XIC B3:0/2 ]-----------------------( OTE O:2/1 )----|
      Yellow_Step                          Yellow_Light
```

Yellow Light O:2/1 is energized while Yellow_Step is active.

8.11 Rung 0010 - Yellow Timer

```
|----[ XIC B3:0/2 ]-----------------------[ TON T4:2 ]----|
      Yellow_Step                         Yellow_Timer   PRE: 500   Time Base: 0.01
```

T4:2 times the Yellow state for 5 seconds.

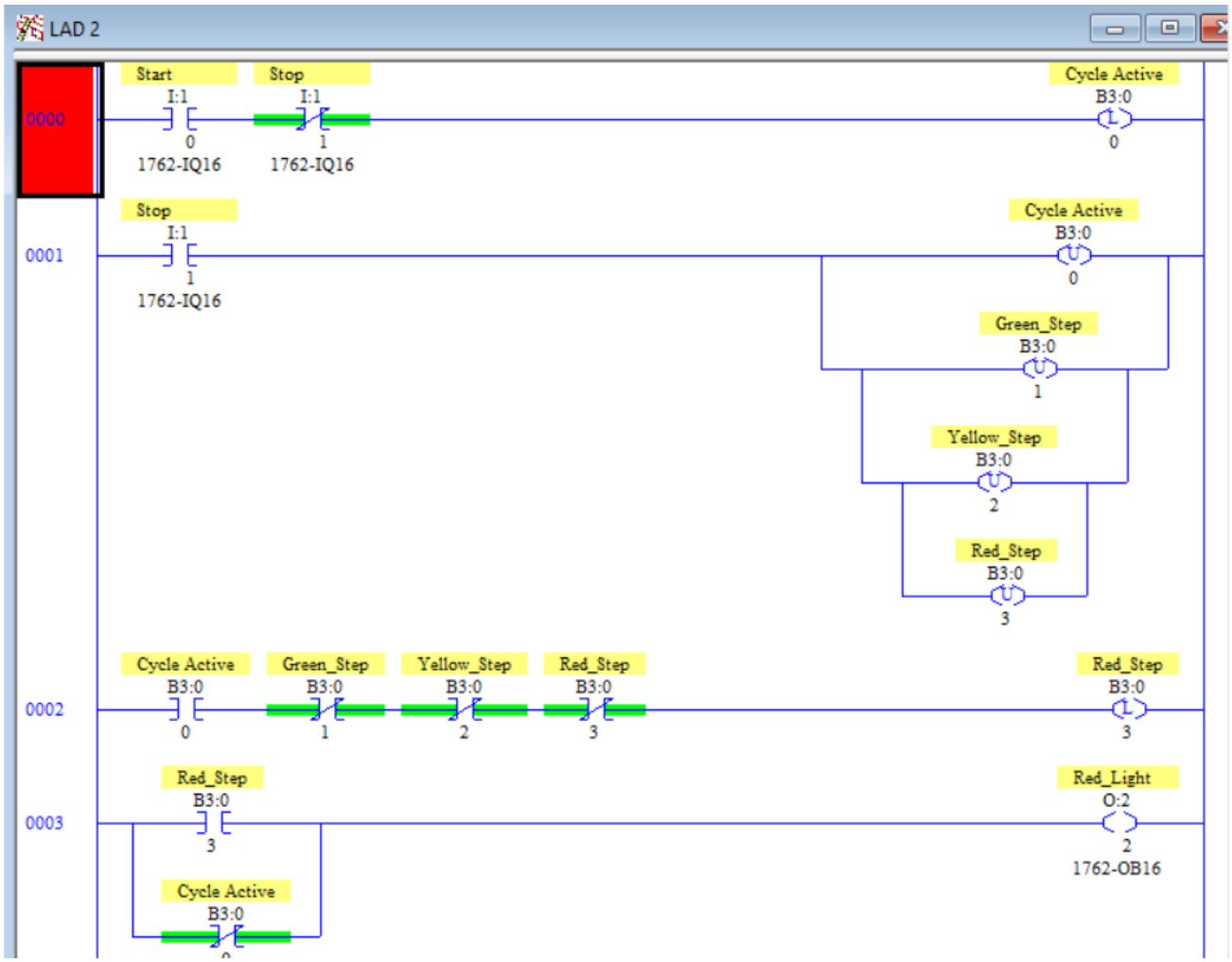
8.12 Rung 0011 - Transition Yellow to Red

```
|----[ XIC T4:2/DN ]----------------------+----( OTU B3:0/2 )----|
       Yellow_Timer_Done                  |      Yellow_Step
                                          +----( OTL B3:0/3 )----|
                                                 Red_Step
```

When Yellow Timer completes, Yellow_Step is cleared and Red_Step is latched, restarting the cycle.

8.13 Actual RSLogix Implementation

The three figures below are the current RSLogix screenshots used as the technical source of truth for this report.



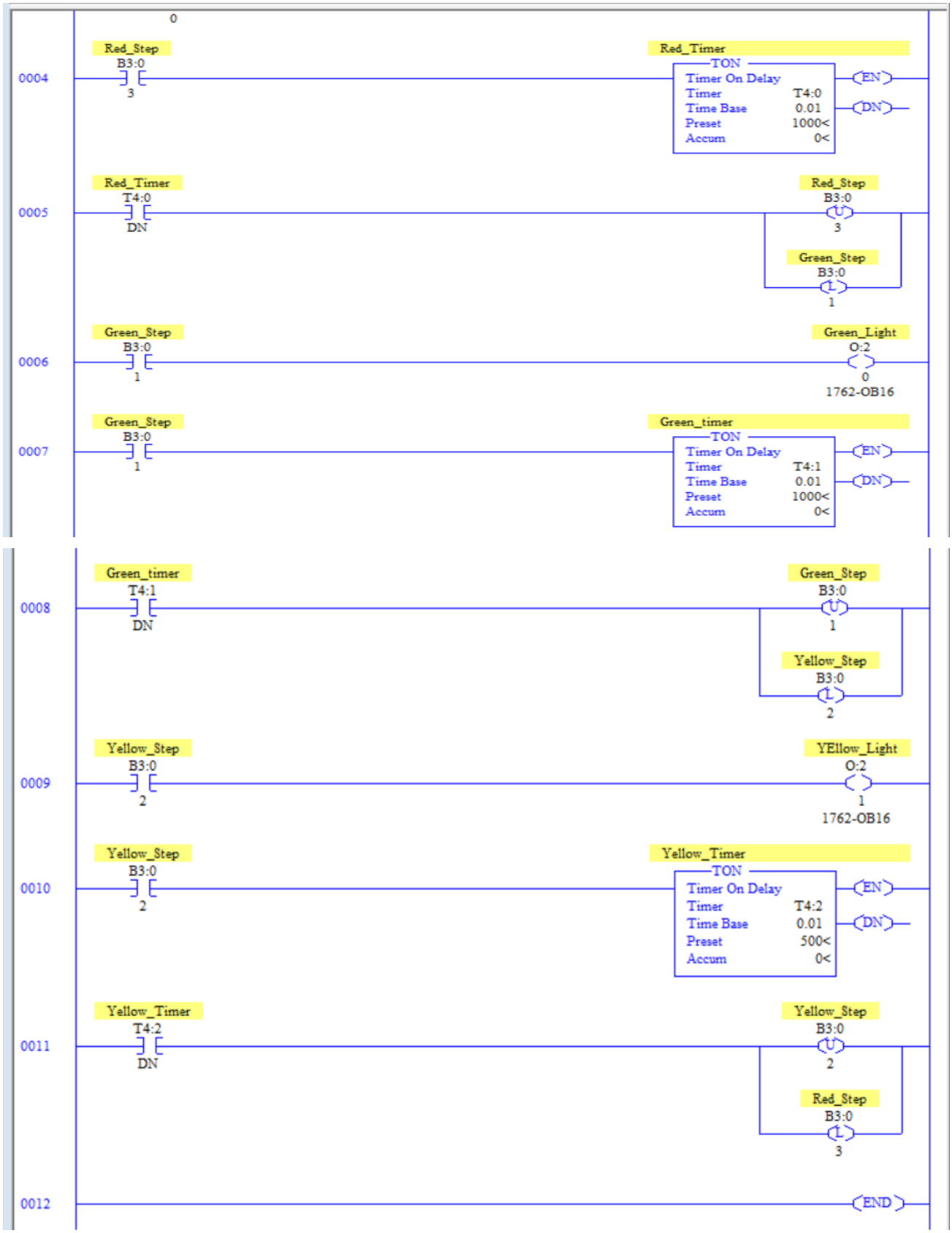


Figure P8.1 - RSLogix Traffic Light Logic

9. Sequence of Operation

Step 1 - Stopped / Default State: Cycle_Active is false and Red Light O:2/2 is ON through the XIO Cycle_Active branch.

Step 2 - Start Command: Start I:1/0 latches Cycle_Active while Stop is inactive.

Step 3 - Red Initialized: With no active step, Red_Step is latched.

Step 4 - Red State: Red Light remains ON and T4:0 times for 10 seconds.

Step 5 - Red to Green: T4:0/DN clears Red_Step and latches Green_Step.

Step 6 - Green State: Green Light turns ON and T4:1 times for 10 seconds.

Step 7 - Green to Yellow: T4:1/DN clears Green_Step and latches Yellow_Step.

Step 8 - Yellow State: Yellow Light turns ON and T4:2 times for 5 seconds.

Step 9 - Yellow to Red: T4:2/DN clears Yellow_Step and latches Red_Step.

Step 10 - Continuous Cycle: The Red -> Green -> Yellow -> Red sequence repeats while Cycle_Active remains true.

Step 11 - Stop Command: Stop clears Cycle_Active and all step bits.

Step 12 - Return to Default Red: With Cycle_Active false, the Red Light remains ON.

10. Testing & Validation

The current ladder screenshots represent the final tested Version 1 implementation. The tests below verify the actual sequence and are recorded as PASS.

Test ID	Test Action / Condition	Expected Result	Observed Result	Status
T-01	Cycle stopped	Cycle_Active false; Red ON; Green/Yellow OFF	Default Red condition confirmed	PASS
T-02	Press Start with Stop inactive	Cycle_Active latches	Cycle_Active latched	PASS
T-03	Cycle starts with no active step	Red_Step initializes	Red_Step latched	PASS
T-04	Red state active for 10 s	Red ON; T4:0 times to done	Red timing completed	PASS
T-05	T4:0 done	Red clears; Green latches	Red-to-Green transition completed	PASS
T-06	Green state active for 10 s	Green ON; T4:1 times to done	Green timing completed	PASS
T-07	T4:1 done	Green clears; Yellow latches	Green-to-Yellow transition completed	PASS
T-08	Yellow state active for 5 s	Yellow ON; T4:2 times to done	Yellow timing completed	PASS
T-09	T4:2 done	Yellow clears; Red latches	Yellow-to-Red transition completed	PASS
T-10	Allow multiple complete cycles	Sequence repeats Red -> Green -> Yellow -> Red	Repeated cyclic operation confirmed	PASS
T-11	Press Stop during any active state	Cycle_Active and all step bits clear	Cycle and steps cleared	PASS
T-12	After Stop	Red ON through default branch	Red default indication restored	PASS

Validation Summary

Testing confirmed the complete cyclic sequence, the 10-second Red and Green durations, the 5-second Yellow duration, the correct timer-driven transitions, and the stopped/default Red behavior.

Stop successfully cleared Cycle_Active and all sequence bits from any active state. All twelve defined Version 1 functional tests passed.

The validation represents functional PLC program testing within the project environment, not traffic-signal certification, public-road commissioning, or safety validation.

11. Results

The completed Traffic Light Controller successfully executed the Red -> Green -> Yellow -> Red sequence using internal state bits and dedicated timers.

Red and Green each operated for 10 seconds, Yellow operated for 5 seconds, and each timer Done bit transferred control to the next state.

The sequence repeated continuously while Cycle_Active remained true. Activating Stop cleared the cycle and step bits and returned the system to the defined Red stopped/default state.

All twelve defined functional tests passed.

12. Challenges & Troubleshooting

A useful troubleshooting path for this project is: Cycle_Active -> Active Step Bit -> Light Output -> Step Timer -> Timer Done -> Next Step Transition.

Symptom	Possible Cause	Diagnostic Check	Corrective Action
Cycle does not start	Start/Stop conditions do not latch Cycle_Active	Monitor I:1/0, I:1/1, B3:0/0	Verify Rung 0000
Red step does not initialize	Another step remains active or Cycle_Active is false	Monitor B3:0/0 through B3:0/3	Verify Rung 0002 conditions
Red Light is OFF while stopped	Default XIO Cycle_Active branch not true	Monitor B3:0/0 and O:2/2	Verify Rung 0003 branch
Red does not transition to Green	T4:0/DN not true	Monitor T4:0.ACC and T4:0/DN	Verify Red timer and Rung 0005
Green does not transition to Yellow	T4:1/DN not true	Monitor T4:1.ACC and T4:1/DN	Verify Green timer and Rung 0008
Yellow does not transition to Red	T4:2/DN not true	Monitor T4:2.ACC and T4:2/DN	Verify Yellow timer and Rung 0011
Wrong light duration	Timer preset or time base incorrect	Check T4:0, T4:1, T4:2	Verify 1000/1000/500 at 0.01 s
Stop does not return to Red default	Cycle/step bits not all cleared	Monitor Rung 0001 and O:2/2	Verify OTU branches and Red default branch

This project reinforced that state-based troubleshooting starts by identifying which step bit is active. Once the active state is known, the associated output, timer, and transition rung can be checked in order.

13. Lessons Learned

This project helped me understand how internal bits can represent explicit operating states in a repeating sequence.

I learned how a dedicated timer can control the duration of each state and how timer Done bits can move the sequence to the next state.

The Stop rung demonstrated the value of clearing all sequence bits together so the program returns to a known condition.

The Red default branch also showed how an output can be intentionally defined for the stopped condition rather than simply allowing all outputs to turn OFF.

14. Industrial Relevance

Timed state sequencing is used throughout industrial automation, including machine cycles, staged process operations, warning sequences, and equipment coordination.

This project uses a traffic-light example to demonstrate a general control pattern: define states, control outputs from the active state, time each state, and transition predictably to the next state.

The Version 1 program is an educational PLC sequence. It is not intended for public-road traffic control and does not include certified traffic-controller hardware, conflict monitoring, redundant safety systems, pedestrian phases, opposing-direction coordination, flashing modes, or regulatory timing requirements.

The engineering value of the project is therefore the PLC sequence-control method rather than direct deployment as a real traffic-signal controller.

15. Transition to Industrial Version (Version 2)

Version 2 can retain the state-based sequencing concept while improving architecture, diagnostics, operating modes, and fault handling.

Version 2 Improvement	Engineering Benefit
Explicit Run Request / System Ready	Separates operator request from sequence execution
Auto/Manual modes	Supports normal automatic sequence and controlled maintenance
Mode-conflict detection	Detects invalid simultaneous mode commands
State validation	Detects impossible or conflicting step conditions
Output conflict checks	Prevents incompatible output combinations
Timer/setpoint organization	Improves maintainability of state durations
Fault latch and controlled reset	Provides defined recovery from abnormal conditions
Startup initialization	Guarantees a known state after restart
Manual lamp test / diagnostics	Supports maintenance verification
HMI state display	Shows active state, timers, outputs, alarms, and faults
Cycle counters / diagnostics	Supports operating history and troubleshooting
Watchdog / transition timeout logic	Detects sequence states that fail to advance

A stronger Version 2 architecture could become: Start Request -> System Ready -> Initialize Safe State -> Red -> Green -> Yellow -> Red, with mode handling, state validation, diagnostics, and controlled fault recovery around the sequence.

The Version 2 design should preserve the clarity of the current state-based logic while making abnormal-condition handling explicit.

16. Project Summary

The Traffic Light Controller introduced cyclic state-based sequencing using internal bits, timers, and discrete outputs.

Cycle_Active B3:0/0 enabled the sequence. Red_Step B3:0/3, Green_Step B3:0/1, and Yellow_Step B3:0/2 represented the active states. T4:0 and T4:1 provided 10-second Red and Green periods, while T4:2 provided a 5-second Yellow period.

Timer Done bits transferred the sequence from Red to Green to Yellow and back to Red. The cycle repeated until Stop cleared Cycle_Active and all step bits. When the cycle was stopped, Red Light O:2/2 remained energized through the default-state branch.

All twelve defined functional tests passed. The project strengthened my understanding of cyclic sequencing, state initialization, timer-driven transitions, controlled stop behavior, and default-state output logic.